МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №2
по курсу «Параллельные и распределенные вычисления»

Обработка изображений на GPU. Фильтры.

Выполнил: М. Д. Иванченко
Группа: М8О-310Б-23
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2026

Условие

Цель работы: научиться использовать GPU для обработки изображений.
Использование текстурной памяти и двухмерной сетки потоков.
Вариант задания: вариант 5. Выделение контуров. Метод Робертса.

Программное и аппаратное обеспечение

Графический процессор:

Характеристика	**Значение**
Compute Capability	8.6
Графическая память (Video Memory)	4 ГБ GDDR6
Шина памяти (Memory Interface)	128-бит
Количество мультипроцессоров (SM)	20
Количество ядер CUDA	2560
Количество тензорных ядер (Tensor Cores)	80 (3-го поколения)
Количество RT-ядер (Ray Tracing Cores)	20 (2-го поколения)
Разделяемая память (Shared Memory)	64 КБ
Константная память (Constant Memory)	64 КБ
Регистров на блок (Registers per Block)	255
Макс. количество нитей (Max Threads)	1536
Макс. количество блоков (Max Blocks)	16

Процессор:

Параметр	**Значение**
Модель	Intel Core i7-12650H (12-е поколение)
Всего ядер	10 (6 производительных + 4 энергоэффективных)
Всего потоков	16

Параметр	Значение
Базовая частота	2.30 ГГц
Макс. частота Turbo	До 4.70 ГГц
Кэш L3 (Smart Cache)	24.0 МБ
Кэш L2	8.8 МБ
Кэш L1	800 КБ
Техпроцесс	Intel 7 (10 нм Enhanced)
Виртуализация	Включена (VT-x / VT-d)

Оперативная память: 32 ГБ.
Объем жёсткого диска: 954 ГБ.
OS: Windows 10.
IDE: Microsoft Visual Studio Community 2022.
Compiler: NVIDIA CUDA Compiler.

Метод решения

Программа реализует детектор границ Робертса на GPU с использованием текстур CUDA. Входное изображение загружается из бинарного файла в формате uchar4 (RGBA), копируется в текстуру на устройстве. Ядро kernel для каждого пикселя берёт четыре соседних текселя с нормализованными координатами, переводит их в яркость, вычисляет диагональные разности и евклидову норму, формируя изображение границ. Используется текстурная выборка с зеркальной адресацией и точечной фильтрацией. Результат сохраняется в аналогичном бинарном формате.

Описание программы

Код реализован в одном файле и не разделён на отдельные модули. В нём используются стандартные типы CUDA: uchar4 для хранения RGBA-пикселей (четыре байта), cudaTextureObject_t для работы с текстурой, а также dim3 для задания размерности сетки и блоков. Основные функции: main (организует ввод-вывод и управление памятью) и ядро kernel (выполняется на GPU). Вспомогательный макрос CSC служит для отлова ошибок CUDA.

Ядро kernel вычисляет градиент яркости для каждого пикселя, реализуя простейший детектор границ (напоминающий оператор Робертса). Оно принимает текстурный объект tex (исходное изображение), указатель на выходной буфер out и размеры w, h. В двойном цикле, покрывающем все пиксели с учётом шага по сетке, для каждой точки (x, y) берутся четыре соседних текселя: p00 (левый верхний), p01 (левый

нижний), p10 (правый верхний), p11 (правый нижний). Их компоненты RGB преобразуются в яркость по формуле Y = 0.299*R + 0.587*G + 0.114*B. Затем вычисляются разности gx = Y11 - Y00 и gy = Y10 - Y01, после чего находится евклидова норма sqrt(gx^2 + gy^2). Полученная величина записывается во все цветовые каналы выходного пикселя (получается полутоновое изображение границ), а альфа-канал сохраняется неизменным.

Функция main считывает из входного бинарного файла размеры w, h и массив данных uchar4. На устройстве создаётся массив cudaArray для текстуры, куда копируются пиксели, и формируется текстурный объект с адресацией Mirror и точечной фильтрацией. По завершении работы результат копируется обратно на хосте и сохраняется в выходной файл в том же формате (ширина, высота, затем пиксели uchar4). В конце все ресурсы (текстура, массив, память на устройстве и хосте) освобождаются.

Результаты

Время GPU:

Размер изображения	**Конфигурация**	**Время GPU (миллисекунды, мс)**
128 × 128 (16 Кп)	<<1, 32>>	0.067
	<<8×8, 16×16>>	0.055
	<<4×4, 32×32>>	0.062
1024 × 1024 (1 М)	<<1, 32>>	0.89
	<<64×64, 16×16>>	0.31
	<<32×32, 32×32>>	0.28
2048 × 2048 (4 М)	<<1, 32>>	3.52
	<<128×128, 16×16>>	1.08
	<<64×64, 32×32>>	0.96
4096 × 4096 (16 М)	<<1, 32>>	14.6
	<<256×256, 16×16>>	4.41
	<<128×128, 32×32>>	3.87
8192 × 8192 (64 М)	<<1, 32>>	59.3
	<<512×512, 16×16>>	17.8
	<<256×256, 32×32>>	16.2

Время CPU:

Размер изображения	**Время (миллисекунды, мс)**
128 × 128 (16 Кп)	0.018
1024 × 1024 (1 М)	8.7
2048 × 2048 (4 М)	35.1
4096 × 4096 (16 М)	142
8192 × 8192 (64 М)	568

Рисунок 1. Входное изображение

Рисунок 2. Выходное изображение

Выводы

Реализованный на GPU оператор Робертса предназначен для выделения границ изображений в различных системах, таких как видеонаблюдение, навигация мобильных роботов, медицинская диагностика и предварительная обработка перед более сложными алгоритмами компьютерного зрения. Благодаря своей простоте – вычисление диагональных разностей яркости и евклидовой нормы – алгоритм идеально подходит для массового параллелизма: каждый пиксель обрабатывается независимо. Сложность программирования средней тяжести: требуется уверенное управление памятью CUDA, создание текстурных объектов и настройка сетки блоков. Основные проблемы возникли с выбором оптимального размера блоков, с краевой адресацией (пришлось применить зеркальный режим, чтобы избежать выхода за пределы) и с преобразованием пиксельных координат в нормализованные.

Сравнение результатов на разных размерах изображений показывает, что для малых картинок (менее 0,1 Мп) время GPU превышает время CPU из-за накладных расходов на передачу данных, запуск ядра и синхронизацию. Однако уже на размере около 1 Мп (например, 1024×1024) правильная конфигурация блоков даёт выигрыш в 2-3 раза, а на 64 Мп (8192×8192) ускорение становится еще больше. Это объясняется тем, что ядра GPU работают параллельно, а текстуры эффективно кешируют выборки соседних пикселей. Наихудшая конфигурация <<1,32>> приводит к недозагрузке GPU и почти линейному росту времени с размером, тогда как оптимальное число блоков (например, 256×256 по 16×16 нитей) позволяет скрыть задержки доступа к памяти и полностью задействовать все мультипроцессоры. Таким образом, предложенная реализация оправдана для изображений от 1 мегапикселя, а для мелких картинок эффективнее использовать CPU.